Ch. 9 Cheat Sheet
AVL and Red-Black Trees

Why balanced BSTs

Raw BST degrades to $O(n)$ on sorted input. AVL and RB are BSTs that enforce a balance invariant so height stays $O(\log n)$ *worst case*.

AVL: the invariant

For every node n, **balance factor**

$$\mathrm{bf}(n) = h(n.\mathrm{left}) - h(n.\mathrm{right}) \in \{-1, 0, +1\}.$$

Convention: height of empty subtree is -1; single node is 0. AVL worst-case height $\approx 1.44 \log_2(n+2)$.

AVL node

```
struct AVLNode {
    int key;
    int height = 0;      // cached
    AVLNode* left  = nullptr;
    AVLNode* right = nullptr;
    AVLNode* parent = nullptr;
};
```

Update `height` after every structural change: $h = 1 + \max(h_L, h_R)$ using -1 for null children.

Rotations (BST-order-preserving swap)

- **Right-rotate at** y: y's left child x becomes root; y becomes x's right child; x's old right subtree becomes y's new left.
- **Left-rotate**: mirror.

In-order sequence is unchanged. This is the universal balancing primitive.

The four AVL imbalance cases

First unbalanced ancestor z; y = taller child; x = child of y

	shape	fix
	LL (left, left)	right-rotate at z
on the same side:	RR (right, right)	left-rotate at z
	LR (left, right)	left-rotate at y, then right-rotate at z
	RL (right, left)	right-rotate at y, then left-rotate at z

AVL insert

1. BST insert at leaf; new node has height 0.
2. **Retrace**: walk up from the new node, updating heights. At the *first* ancestor with $|\mathrm{bf}| > 1$, apply the matching rotation. One rotation (single or double) is enough – subtree height returns to its pre-insert value.

Cost: $O(\log n)$ descent + $O(\log n)$ retrace = $O(\log n)$.

AVL remove

1. BST remove (two-children $\to$ copy successor key, remove successor – which has ≤ 1 child).
2. Retrace from the removed node's parent. Unlike insert, rebalancing may **cascade**: fixing one level can leave the next one unbalanced.

Red-black: the five rules

1. Every node is red or black.
2. The root is black.
3. A red node's children are black (no two reds in a row).
4. Every null (NIL) leaf is black.
5. Every root-to-NIL path has the same number of black nodes (the *black-height*).

Together $\Rightarrow$ longest path $\leq 2\times$ shortest $\Rightarrow$ height $\leq 2\log_2(n+1)$.

RB insertion fix-up, case summary

New node inserted **red**. If parent is black, done. Else (parent red – rule 3 violated), by uncle:

- **Uncle red** (case 3): recolor parent & uncle black, grandparent red. Recurse at grandparent.
- **Uncle black, zig-zag** (case 4): rotate parent to straighten, then fall into case 5.
- **Uncle black, straight line** (case 5): recolor, rotate grandparent. Done.

Then: root $\to$ black. At most $O(\log n)$ recolorings and $O(1)$ rotations.

RB removal (the hard one)

BST-remove first. Only removing a *black* node is tricky – it breaks rule 5. Track a "double-black" token and fix via six cases (red sibling, black sibling + red-nephew variants, black sibling + black nephews $\to$ push deficit up). Worst case $O(\log n)$ with $O(1)$ amortized rotations.

AVL vs. red-black

	AVL	**red-black**
height	$\leq 1.44\log n$	$\leq 2\log n$
search	faster (tighter)	slightly slower
insert	up to 2 rotations	≤ 2 rotations
remove	may cascade	≤ 3 rotations
used by	specialized DBs	`std::map`, Linux kernel

Rule of thumb: AVL if reads dominate; RB if writes dominate.

Top gotchas

- Forgetting to update cached heights after a rotation.
- Reading balance factor with null children as height 0 instead of -1.
- Applying rotations at the wrong pivot – read the cases as "name from grandparent down."
- Treating remove-retrace as "one rotation fixes it" – it can cascade all the way to the root.
- In RB: forgetting to reset root to black after a case-3 recolor propagates it upward.